

# System Design Problems — Foundational



## In This Edition

---

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>1. Design a URL Shortener (TinyURL / bit.ly)</b>      | <b>2</b>  |
|          | 1.1 1. Requirements                                      | 2         |
|          | 1.2 2. Capacity Estimation                               | 2         |
|          | 1.3 3. API Design                                        | 3         |
|          | 1.4 4. Data Model                                        | 3         |
|          | 1.5 5. High-Level Architecture                           | 4         |
|          | 1.6 6. Deep Dives                                        | 5         |
|          | 1.7 7. Bottlenecks & Scaling                             | 6         |
|          | 1.8 8. Trade-offs Summary                                | 7         |
| <b>2</b> | <b>2. Design a Rate Limiter</b>                          | <b>7</b>  |
|          | 2.1 1. Requirements                                      | 7         |
|          | 2.2 2. Capacity Estimation                               | 7         |
|          | 2.3 3. API Design                                        | 8         |
|          | 2.4 4. Data Model                                        | 8         |
|          | 2.5 5. High-Level Architecture                           | 9         |
|          | 2.6 6. Deep Dives                                        | 10        |
|          | 2.7 7. Bottlenecks & Scaling                             | 12        |
|          | 2.8 8. Trade-offs Summary                                | 12        |
| <b>3</b> | <b>3. Design a News Feed (Facebook/Twitter Timeline)</b> | <b>12</b> |
|          | 3.1 1. Requirements                                      | 12        |
|          | 3.2 2. Capacity Estimation                               | 13        |
|          | 3.3 3. API Design                                        | 13        |
|          | 3.4 4. Data Model                                        | 14        |
|          | 3.5 5. High-Level Architecture                           | 15        |
|          | 3.6 6. Deep Dives                                        | 15        |
|          | 3.7 7. Bottlenecks & Scaling                             | 16        |
|          | 3.8 8. Trade-offs Summary                                | 17        |
| <b>4</b> | <b>4. Design a Chat System (WhatsApp / Messenger)</b>    | <b>17</b> |
|          | 4.1 1. Requirements                                      | 17        |
|          | 4.2 2. Capacity Estimation                               | 18        |
|          | 4.3 3. API Design                                        | 18        |
|          | 4.4 4. Data Model                                        | 19        |
|          | 4.5 5. High-Level Architecture                           | 20        |
|          | 4.6 6. Deep Dives                                        | 21        |
|          | 4.7 7. Bottlenecks & Scaling                             | 23        |
|          | 4.8 8. Trade-offs Summary                                | 24        |
| <b>5</b> | <b>Visual Reference</b>                                  | <b>25</b> |

Complete end-to-end worked designs for a 45–60 min FAANG system-design round. Each section mirrors the structure you should drive in the actual interview.

## 1 1. Design a URL Shortener (TinyURL / bit.ly)

### 1.1 1. Requirements

Clarifying questions to ask the interviewer:

- › What is the expected scale? (DAU, writes per second, reads per second?)
- › Should we support custom aliases (user-defined short codes)?
- › What is the required URL lifetime? Do links expire?
- › Do we need analytics (click counts, geo, referrer)?
- › Should we support link editing or deletion?
- › Is high availability or strong consistency more important?

**Functional Requirements:**

1. Given a long URL, return a unique short URL (e.g., `https://tny.io/aB3kX9`).
2. Redirect a short URL to the original long URL.
3. (Optional) Custom aliases: user can specify the short code.
4. (Optional) URL expiry: links expire after N days.
5. (Optional) Analytics: track click count, geography, referrer.

**Non-Functional Requirements:**

- › 100 ms P99 redirect latency.
- › 99.99% availability (URL shortener is a critical dependency for customers).
- › Short codes must be unique and not predictable/enumerable.
- › The system is **read-heavy** (100:1 read-to-write ratio is typical).

### 1.2 2. Capacity Estimation

**Assumptions:**

- › 500 million URLs shortened per month (writes).
- › 100:1 read-to-write ratio → 50 billion redirects per month.

**Write QPS:**

$500,000,000 / (30 \times 86,400) \approx 193 \text{ writes/sec} \approx \sim 200 \text{ writes/sec}$   
 Peak (3x):  $\approx \sim 600 \text{ writes/sec}$

**Read QPS:**

$50,000,000,000 / (30 \times 86,400) \approx 19,290 \text{ reads/sec} \approx \sim 20,000 \text{ reads/sec}$   
 Peak (3x):  $\approx \sim 60,000 \text{ reads/sec}$

**Storage (URLs, 5 years):**

```
500M writes/month × 12 months × 5 years = 30 billion URLs
Each record ≈ 500 bytes (long URL ~400 B + metadata ~100 B)
Total: 30B × 500 B = 15 TB
```

**Cache (top 20% URLs serve 80% traffic):**

$20,000 \text{ reads/sec} \times 86,400 \text{ sec/day} = 1.728 \text{ billion reads/day}$   
 $20\% \text{ of } 30\text{B URLs} = 6 \text{ billion "hot" URLs} \rightarrow \text{not all fit in RAM}$   
 Pragmatic: cache last 24 hrs of writes =  $500\text{M}/30 \times 500\text{B} \approx 8.3 \text{ GB/day} \rightarrow \sim 10 \text{ GB RAM}$

**Bandwidth:**

Inbound (writes):  $200/\text{sec} \times 500 \text{ B} \approx 100 \text{ KB/s}$   
 Outbound (reads):  $20,000/\text{sec} \times 500 \text{ B} \approx 10 \text{ MB/s}$

**1.3 3. API Design**

```

# Shorten a URL
POST /api/v1/shorten
Request:
{
  "long_url": "https://www.example.com/very/long/path?q=something",
  "custom_code": "my-alias", // optional
  "expires_at": "2027-01-01T00:00Z" // optional
}
Response 201:
{
  "short_url": "https://tny.io/aB3kX9",
  "short_code": "aB3kX9",
  "expires_at": "2027-01-01T00:00Z"
}

# Redirect (the critical read path)
GET /{short_code}
Response 302:
  Location: https://www.example.com/very/long/path?q=something

# Analytics (optional)
GET /api/v1/stats/{short_code}
Response 200:
{
  "clicks": 14982,
  "top_countries": [...],
  "referrers": [...]
}

# Delete
DELETE /api/v1/shorten/{short_code}
Response 204
  
```

**1.4 4. Data Model****SQL vs NoSQL decision:**

- › The primary access pattern is a simple key-value lookup: `short_code` → `long_url`.
- › No complex joins needed; writes are straightforward inserts.
- › **Choice: NoSQL key-value store (Cassandra or DynamoDB) for the URL table** — horizontal scale, high availability, and the access pattern is a perfect fit.
- › A relational DB (PostgreSQL) is used for user/account data where consistency matters.

**URL Table (Cassandra / DynamoDB):**

```

urls

short_code  TEXT      PRIMARY KEY      -- "aB3kX9"
long_url    TEXT      NOT NULL          -- original URL
user_id     UUID
created_at  TIMESTAMP
expires_at  TIMESTAMP          -- NULL = never expires
is_deleted  BOOLEAN  DEFAULT false

```

### Users Table (PostgreSQL):

```

users

user_id     UUID      PRIMARY KEY
email       VARCHAR(255) UNIQUE NOT NULL
api_key     VARCHAR(64) UNIQUE          -- for rate limiting
created_at  TIMESTAMP

```

### Analytics Table (write-optimized, e.g., Cassandra or ClickHouse):

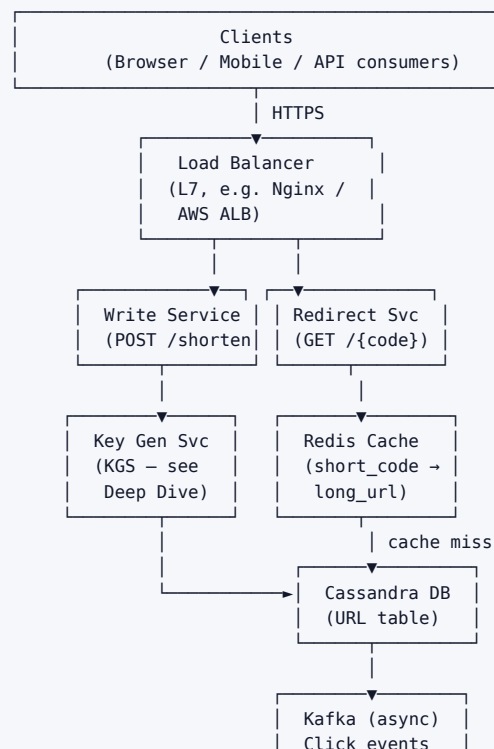
```

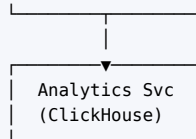
click_events

short_code  TEXT
clicked_at  TIMESTAMP
country     TEXT
referrer    TEXT
user_agent  TEXT
-- Partition key: short_code; Clustering key: clicked_at DESC

```

## 1.5 5. High-Level Architecture





### Component walkthrough:

1. **Client** sends POST /shorten or GET /{code} over HTTPS.
2. **Load Balancer** routes to stateless Write Service or Redirect Service pods.
3. **Redirect Service** checks **Redis** first (cache hit → 302 immediately). On cache miss, reads Cassandra, populates cache, then redirects.
4. **Write Service** calls **KGS** (Key Generation Service) for a pre-generated unique code, stores (code, long\_url) in Cassandra, and warms the cache.
5. **Kafka** asynchronously receives click events; **Analytics Service** batch-writes to ClickHouse for reporting.

## 1.6 6. Deep Dives

### 6.1 Key Generation: How to create short codes?

| Approach                  | How                                                                    | Pros                                      | Cons                                                         |
|---------------------------|------------------------------------------------------------------------|-------------------------------------------|--------------------------------------------------------------|
| <b>MD5 / SHA-256 hash</b> | Hash long URL, take first 6–8 chars                                    | Deterministic (same URL → same code)      | Collisions possible; must check DB on every write            |
| <b>Counter + Base62</b>   | Global atomic counter → encode in Base62                               | No collisions; simple                     | Single counter = SPOF; reveals ordering/volume               |
| <b>Pre-generated KGS</b>  | Dedicated service pre-generates & stores unused keys in a "keys" table | No collision checking at write time; fast | Additional service to maintain; key theft if service crashes |
| <b>UUID</b>               | Generate random UUID, take first 8 chars                               | Simple                                    | High collision probability at scale                          |

**Key Decision: KGS (Key Generation Service) is the recommended approach for large scale.**

KGS design:

- › KGS pre-generates 6-character Base62 codes ( $62^6 \approx 56$  billion unique codes).
- › Stores them in two tables: unused\_keys and used\_keys.
- › Each Write Service instance requests a batch (e.g., 1,000 keys) from KGS on startup, holds them in-memory.
- › KGS marks those keys as "in use" atomically. If a Write Service pod crashes, those keys are wasted but never reused — acceptable loss.
- › KGS itself can be replicated with a leader for key assignment.

### 6.2 Redirect: 301 vs 302?

|                       | 301 Permanent                                 | 302 Temporary                                   |
|-----------------------|-----------------------------------------------|-------------------------------------------------|
| Browser caches?       | Yes — browser won't call TinyURL again        | No — browser calls TinyURL on every click       |
| Analytics accuracy    | <b>Broken</b> — browser bypasses your servers | <b>Accurate</b> — every click hits your servers |
| Load on TinyURL       | Lower after first visit                       | Higher (every click)                            |
| <b>Recommendation</b> | Use only if analytics not needed              | <b>Use 302 for full analytics capability</b>    |

**Interview takeaway:** Always use 302 unless you explicitly trade analytics for reduced server load.

### 6.3 Collision Handling

If using hash-based approach:

1. Compute MD5(long\_url) → take first 7 chars as short\_code.
2. Check if short\_code exists in DB.
3. If collision: append a salt (e.g., increment integer), re-hash, retry.
4. Limit retries to 3; fall back to random generation if exhausted.

With KGS, collisions are impossible — keys are pre-assigned and marked used atomically.

### 6.4 Custom Aliases

- › User provides desired short code (e.g., my-brand).
- › Check availability: `SELECT 1 FROM urls WHERE short_code = 'my-brand'.`
- › If taken → return 409 Conflict.
- › Store with `is_custom = true` flag to distinguish from generated codes.
- › Rate-limit custom alias creation per user to prevent squatting.

### 6.5 URL Expiry

- › Store `expires_at` timestamp in the URL record.
- › On redirect: if `expires_at < NOW()` → return 404 (or 410 Gone).
- › Background cleanup job: delete expired records weekly (keep DB lean).
- › Cache entries: set Redis TTL = `expires_at - NOW()` so expired URLs auto-evict.

## 1.7 7. Bottlenecks & Scaling

**Read hotspot — viral URLs:**

- › A single URL (e.g., link from a viral tweet) can receive millions of reads/sec.
- › Redis handles this well — single key lookup is  $O(1)$  and Redis handles 100k+ ops/sec per node.
- › For extreme cases: replicate the specific key across multiple Redis nodes or use local in-process cache (LRU with 1000 entries) in each Redirect Service pod.

**Write scaling:**

- › Writes at ~200/sec are trivially handled. Cassandra scales horizontally; add nodes as needed.
- › KGS: single KGS with a read replica for HA is sufficient at this scale. Key generation is CPU-light.

**DB sharding:**

- › Cassandra partitions by `short_code` automatically (consistent hashing). No manual sharding needed.
- › If using SQL: shard by `short_code` modulo N (hash partitioning). Use a consistent hash ring to avoid resharding pain.

**Cache eviction:**

- › Redis with LRU policy. Set max memory = 10–20 GB.
- › TTL on each entry = `min(expires_at, 24 hrs)` to prevent stale data.

**Failure handling:**

- › Redis down → fall through to Cassandra (latency increases but system stays up).
- › KGS down → Write Service uses its in-memory key batch until KGS recovers; alert if batch exhausted.
- › Cassandra node down → Cassandra replication factor 3 (RF=3) ensures reads/writes still succeed with 2 nodes up.

**Geographic distribution:**

- › Deploy read replicas of Cassandra in multiple regions.
- › Use GeoDNS to route users to nearest Redirect Service cluster.
- › URL writes go to a primary region; replicated asynchronously to others.

## 1.8 8. Trade-offs Summary

| Decision           | Choice Made                       | Alternative            | Reason                                             |
|--------------------|-----------------------------------|------------------------|----------------------------------------------------|
| Key generation     | <b>KGS (pre-generated)</b>        | Hash + collision check | No collision risk; predictable latency             |
| DB for URLs        | <b>Cassandra (NoSQL)</b>          | PostgreSQL             | Write scalability; simple key-value access pattern |
| Redirect type      | <b>302 Temporary</b>              | 301 Permanent          | Enables accurate analytics                         |
| Cache              | <b>Redis (LRU)</b>                | Memcached              | Rich data types; TTL support; persistence          |
| Analytics pipeline | <b>Kafka → ClickHouse (async)</b> | Sync write to SQL      | Decouples write path; handles bursty click events  |
| Custom aliases     | <b>Supported (rate-limited)</b>   | Not supported          | Product differentiator; must prevent abuse         |

## 2. Design a Rate Limiter

## 2.1 1. Requirements

### Clarifying questions to ask the interviewer:

- › Where should the rate limiter live? Client-side, API gateway, or per-service middleware?
- › What are we rate-limiting on? Per user? Per IP? Per API key? Per endpoint?
- › What happens when a limit is exceeded — hard reject (429) or soft throttle (queue)?
- › Do we need distributed rate limiting (multiple servers sharing state)?
- › What are the latency requirements? Rate limit check must not add significant overhead.
- › Do rules need to be configurable at runtime without deployments?

### Functional Requirements:

1. Limit the number of requests a client (user/IP/API key) can make in a time window.
2. Return HTTP 429 Too Many Requests when limit is exceeded, with `Retry-After` header.
3. Support multiple limits (e.g., 100 req/min AND 1,000 req/hr for same client).
4. Rules are configurable without service restarts.

### Non-Functional Requirements:

- › Rate limit check must add  $< 5$  ms P99 latency overhead.
- › Highly available — rate limiter failure should fail open (allow traffic) not fail closed (block everything).
- › Works across multiple servers (distributed, consistent limits).
- › Accurate enough — exact precision not required; small over-limit acceptable.

## 2.2 2. Capacity Estimation

**Assumptions:**

- 10 million DAU, each making ~100 requests/day → 1 billion requests/day.
- Rate limit state must be checked for every request.

### QPS to rate limiter:

1,000,000,000 / 86,400  $\approx$  11,574 req/sec  $\approx$  ~12,000 checks/sec  
Peak (3x):  $\approx$  ~36,000 checks/sec



**Memory for rate limit counters:**

Strategy: sliding window counter per user per endpoint, 1-hr window.  
 $10\text{M users} \times 10 \text{ endpoints} \times (\text{counter} + \text{timestamp}) \approx 10\text{M} \times 10 \times 20 \text{ bytes} = 2 \text{ GB}$   
 → Fits comfortably in a Redis cluster.

**Bandwidth:**

- › Rate limit check: small — just a counter lookup and increment.
- › Each Redis call  $\approx$  microseconds; no large data transfer.

**2.3 3. API Design**

The rate limiter is typically middleware, not a standalone API. But if exposing as a service:

```
# Check and record a request (called by API gateway)
POST /rate-limit/check
Request:
{
  "client_id": "user:1234",
  "endpoint": "/api/v1/posts",
  "timestamp": 1718000000
}
Response 200 (allowed):
{
  "allowed": true,
  "remaining": 42,
  "reset_at": 1718003600
}
Response 429 (throttled):
{
  "allowed": false,
  "retry_after": 58,           // seconds until window resets
  "limit": 100,
  "window": "1m"
}

# Update rules (admin)
PUT /rate-limit/rules/{client_id}
Request:
{
  "rules": [
    { "window": "1m", "limit": 100 },
    { "window": "1h", "limit": 1000 }
  ]
}
```

**Response headers to include on every API response:**

```
X-RateLimit-Limit:    100
X-RateLimit-Remaining: 42
X-RateLimit-Reset:    1718003600
Retry-After:          58           // only on 429
```

**2.4 4. Data Model**

**Redis data structures (no SQL needed — state is ephemeral):**

**Token Bucket / Fixed Window counter:**

```

Key: "rl:{client_id}:{endpoint}:{window_start}"
Value: integer (request count)
TTL: window duration (e.g., 60s for 1-min window)

```

### Sliding Window Log:

```

Key: "rl:{client_id}:{endpoint}"
Type: Redis Sorted Set
Member: request_id (UUID)
Score: timestamp (epoch ms)
TTL: window duration

```

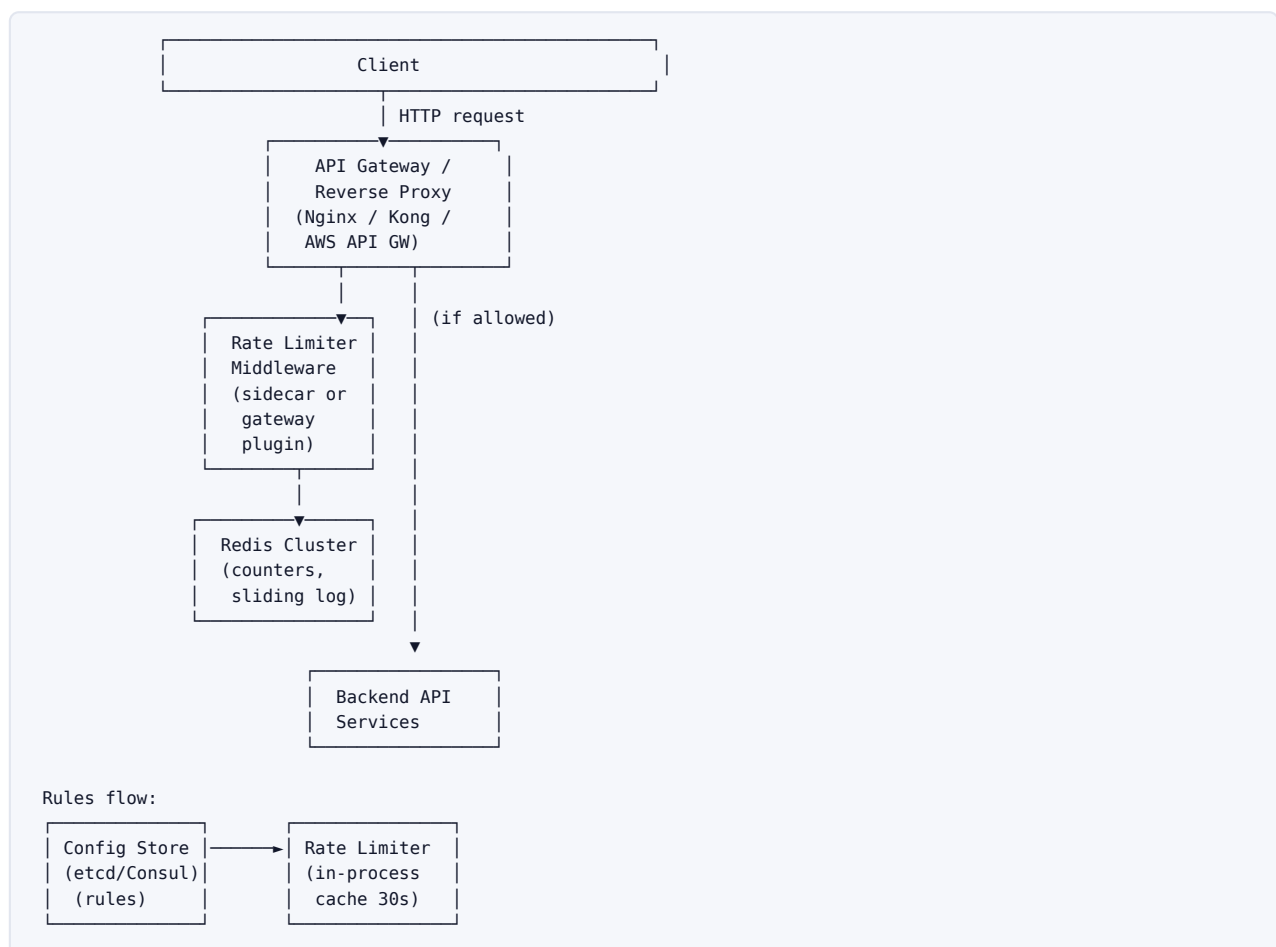
### Rules config (read rarely, cache in-process):

```

# Redis Hash or a config store (Consul / etcd)
Key: "rule:{client_id}"
Value: JSON { "rules": [{ "window": "1m", "limit": 100 }, ... ] }

```

## 2.5 5. High-Level Architecture



### Component walkthrough:

1. Every request hits the **API Gateway**.

2. Gateway invokes **Rate Limiter Middleware** (same process or sidecar).
3. Middleware reads the rule for (`client_id`, `endpoint`) from its **in-process cache** (refreshed every 30s from etcd/Consul).
4. Middleware atomically increments/checks the counter in **Redis**.
5. If allowed: forward request to **Backend Service**.
6. If denied: return 429 immediately with Retry-After.

## 2.6 6. Deep Dives

### 6.1 Algorithm Comparison

| Algorithm                     | How it works                                                        | Pros                            | Cons                                                         | Best for                        |
|-------------------------------|---------------------------------------------------------------------|---------------------------------|--------------------------------------------------------------|---------------------------------|
| <b>Fixed Window Counter</b>   | Reset counter every N seconds                                       | Simple; O(1) memory             | Boundary burst: $2 \times$ limit possible at window edge     | Simple internal limits          |
| <b>Sliding Window Log</b>     | Store timestamp of every request in sorted set; count within window | Accurate                        | O(n) memory per user (n = requests in window)                | Low-volume, high-accuracy needs |
| <b>Sliding Window Counter</b> | Blend current + previous window counts weighted by overlap          | Near-accurate, O(1) memory      | Slight inaccuracy (~0.003% over-limit)                       | <b>Best general choice</b>      |
| <b>Token Bucket</b>           | Bucket of tokens refilled at rate R; request consumes one           | Allows bursts up to bucket size | Harder to reason about exact limits; burst may confuse users | APIs that allow bursting        |
| <b>Leaky Bucket</b>           | Queue requests; process at constant rate                            | Smooth output rate              | Queue introduces latency; bursty requests get delayed        | Network traffic shaping         |

**Key Decision: Sliding Window Counter for most API rate limiting. Token Bucket when you want to allow short bursts.**

### 6.2 Sliding Window Counter --- Implementation Detail

```
Window = 1 minute.
Current minute bucket: starts at :00
Previous minute bucket: started at :-60

count = prev_bucket_count * (overlap / window) + curr_bucket_count

Example at :45 into the current minute:
overlap = 60 - 45 = 15 seconds (15/60 = 25% of previous window still counts)
count = prev_count * 0.25 + curr_count

If count > limit → reject.
```

Redis implementation:

```
-- Lua script (atomic)
local key_curr = KEYS[1] -- "rl:user1:1718000060"
local key_prev = KEYS[2] -- "rl:user1:1718000000"
local limit    = tonumber(ARGV[1])
local now      = tonumber(ARGV[2])
local window   = tonumber(ARGV[3])
local window_start = now - (now % window)
local overlap  = window - (now - window_start)
```

```

local curr_count = tonumber(redis.call('GET', key_curr) or 0)
local prev_count = tonumber(redis.call('GET', key_prev) or 0)
local weighted  = prev_count * (overlap / window) + curr_count

if weighted >= limit then
    return 0 -- rejected
end
redis.call('INCR', key_curr)
redis.call('EXPIRE', key_curr, window * 2)
return 1 -- allowed

```

### 6.3 Where to Place the Rate Limiter?

| Placement                           | Pros                                            | Cons                                                               |
|-------------------------------------|-------------------------------------------------|--------------------------------------------------------------------|
| <b>Client-side</b>                  | Zero server load                                | Easily bypassed; untrustworthy                                     |
| <b>API Gateway</b>                  | Single enforcement point; language-agnostic     | Gateway becomes a bottleneck; less context                         |
| <b>Service middleware</b>           | Per-service granularity; business context       | Duplicated logic across services; harder to change rules centrally |
| <b>Dedicated Rate Limit Service</b> | Centralized rules; all services call it         | Extra network hop; additional failure point                        |
| <b>Recommended: Gateway + Redis</b> | Rules centralized in Redis; enforced at gateway | Need to handle Redis failure gracefully                            |

### 6.4 Distributed Rate Limiting --- Race Conditions

**Problem:** Two requests arrive simultaneously on different servers. Both read count=99 (limit=100), both increment to 100, both allow. Result: 2 requests slip through over limit.

**Solutions:**

1. **Redis Lua scripts (recommended):** Atomic read-increment-check in a single Lua script. Redis is single-threaded; no race condition possible.
2. **Redis INCR + EXPIRE pattern:**

```

count = INCR(key)
if count == 1: EXPIRE(key, window) -- set TTL only on first request
if count > limit: return 429

```

Problem: race between INCR and EXPIRE if server crashes. Mitigate with SET key 0 EX window NX on first call.

3. **Sticky sessions / consistent hashing:** Route same client\_id to same rate limiter node. Eliminates distributed counting but adds routing complexity and uneven load.

**Key Decision: Use Redis Lua scripts for atomicity. No locks, no transactions overhead.**

### 6.5 Fail-Open vs Fail-Closed

- **Fail-open (recommended):** If Redis is down, allow all requests. Availability > perfect rate limiting.
- **Fail-closed:** If Redis is down, reject all requests. Too severe for most use cases — takes down your API.
- **Implementation:** wrap Redis call in try-catch; on exception, log and allow the request, alert on-call.

## 2.7 7. Bottlenecks & Scaling

### Redis as single point of failure:

- › Use Redis Cluster (sharded) — each client\_id maps to a shard via consistent hashing.
- › Redis Sentinel for HA (automatic failover).
- › Replication lag between primary and replica: acceptable (small over-limit possible).

#### Redis latency:

- › Redis P99  $\approx$  0.5–1 ms in-datacenter. Total rate limit overhead  $< 2$  ms.
- › If Redis is co-located on same host (Redis sidecar per gateway node): P99  $< 0.2$  ms.

#### Rule changes at scale:

- › Store rules in etcd/Consul. Rate limiter middleware subscribes to watch-notifications.
- › In-process cache with 30s TTL: small window where old rules apply after update.

#### Hotspots (one user generating massive traffic):

- › Rate limiter itself handles this well — a single Redis key gets hammered, but Redis is designed for this.
- › Use local in-process counter as first layer (no Redis call for clearly non-bursting clients), then Redis for enforcement.

## 2.8 8. Trade-offs Summary

| Decision      | Choice Made                    | Alternative          | Reason                                                           |
|---------------|--------------------------------|----------------------|------------------------------------------------------------------|
| Algorithm     | <b>Sliding Window Counter</b>  | Token Bucket         | Near-exact accuracy; O(1) memory; prevents boundary bursts       |
| State store   | <b>Redis Cluster</b>           | In-process memory    | Distributed consistency across servers; fast enough              |
| Placement     | <b>API Gateway middleware</b>  | Dedicated service    | Avoids extra network hop; centralizes enforcement                |
| Atomicity     | <b>Redis Lua scripts</b>       | Redis WATCH/MULTI    | Simpler; no retry logic needed                                   |
| Failure mode  | <b>Fail-open</b>               | Fail-closed          | Availability $>$ perfect accuracy for most systems               |
| Rules storage | <b>etcd + in-process cache</b> | DB query per request | Rules rarely change; in-process cache eliminates lookup overhead |

## 3 3. Design a News Feed (Facebook/Twitter Timeline)

### 3.1 1. Requirements

#### Clarifying questions to ask the interviewer:

- › Is this a "friends" model (Facebook — bidirectional) or "followers" model (Twitter — unidirectional)?
- › What is the feed ordering — chronological or ranked by relevance?
- › What's the expected social graph size? Max followers per user?
- › Do we support stories (ephemeral) vs. posts (permanent)?
- › Is the feed pre-generated (push) or assembled on request (pull)?
- › Do we need to support ads injection in the feed?

#### Functional Requirements:

1. User can create a post (text, image, video link).
2. User sees a feed of posts from people they follow, ordered by recency or ranking.

3. Feed updates in near-real-time (within seconds of post creation).
4. Pagination: load N posts, scroll for more.
5. Users can like, comment, share (out of scope for this design; focus on feed generation).

#### Non-Functional Requirements:

- › 500 million DAU (Facebook scale).
- › Feed load latency < 200 ms.
- › Post creation propagates to followers within 5 seconds (eventual consistency is OK).
- › Users can have up to 5,000 friends (Facebook) or millions of followers (Twitter celebrities).

## 3.2 2. Capacity Estimation

### Assumptions:

- › 500 million DAU.
- › Each user views feed 5 times/day → 2.5 billion feed reads/day.
- › Each user creates 1 post every 5 days → 100 million new posts/day.
- › Average friends/followers: 300.

#### Post write QPS:

$100,000,000 / 86,400 \approx 1,157 \text{ writes/sec} \approx \sim 1,200 \text{ posts/sec}$   
 Peak (5×):  $\approx \sim 6,000 \text{ posts/sec}$

#### Feed read QPS:

$2,500,000,000 / 86,400 \approx 28,935 \text{ reads/sec} \approx \sim 29,000 \text{ reads/sec}$   
 Peak (3×):  $\approx \sim 87,000 \text{ reads/sec}$

#### Fanout writes (push model — writing to all follower feeds on post):

$1,200 \text{ posts/sec} \times 300 \text{ avg followers} = 360,000 \text{ feed writes/sec}$   
 Celebrity (10M followers): 1 post → 10M writes (burst)

#### Storage:

Posts:  $100\text{M/day} \times 1 \text{ KB (text)} = 100 \text{ GB/day text}$   
 Images/videos: separate blob store (S3)  
 Feed cache:  $500\text{M users} \times 20 \text{ cached posts} \times 1 \text{ KB} = 10 \text{ TB} \rightarrow \text{too big for RAM}$   
 → Cache only active users (10% daily active at peak):  
 $50\text{M users} \times 20 \text{ posts} \times 1 \text{ KB} = 1 \text{ TB} \rightarrow \text{feasible with Redis cluster}$

## 3.3 3. API Design

```

# Create a post
POST /api/v1/posts
Auth: Bearer token
Request:
{
  "content": "Hello world!",
  "media_urls": ["https://cdn.example.com/img/abc.jpg"],
  "visibility": "friends" // or "public", "private"
}
Response 201:
{

```

```

    "post_id":    "p_9x3kA2",
    "created_at": "2026-06-14T10:00:00Z"
  }

# Get news feed
GET /api/v1/feed?limit=20&cursor=<opaque_cursor>
Auth: Bearer token
Response 200:
{
  "posts": [
    {
      "post_id":    "p_9x3kA2",
      "author_id":  "u_4f2k",
      "author_name": "Alice",
      "content":    "Hello world!",
      "media_urls": [...],
      "like_count": 142,
      "comment_count": 7,
      "created_at": "2026-06-14T10:00:00Z"
    },
    ...
  ],
  "next_cursor": "<opaque_cursor>"
}

# Get a user's profile posts
GET /api/v1/users/{user_id}/posts?limit=20&cursor=<cursor>
Response 200: { "posts": [...], "next_cursor": "..."}

```

### 3.4 4. Data Model

#### SQL vs NoSQL:

- › **Posts table:** Write-once, read-many. High read QPS. Shard by `author_id`. → NoSQL (Cassandra) or sharded PostgreSQL.
- › **Social graph (follows):** Relationship lookups ("who does user X follow?"). → Graph DB (Neo4j) or sharded SQL; at Facebook scale: custom TAO (graph store).
- › **Feed cache:** Redis sorted sets (score = post timestamp).
- › **User data:** Relational (PostgreSQL) — low write volume, complex queries.

#### Posts Table (Cassandra — partition by author, cluster by time):

```

posts

author_id  UUID      PARTITION KEY
post_id    TIMEUUID   CLUSTERING KEY (DESC)  -- newest first
content    TEXT
media_urls LIST<TEXT>
like_count COUNTER
created_at TIMESTAMP

```

#### Social Graph Table (Cassandra — two tables for O(1) lookups):

```

follows_by_follower      -- "who does user X follow?"

follower_id  UUID  PARTITION KEY
followee_id  UUID  CLUSTERING KEY

follows_by_followee      -- "who follows user X?"

```

```

followee_id  UUID  PARTITION KEY
follower_id  UUID  CLUSTERING KEY

```

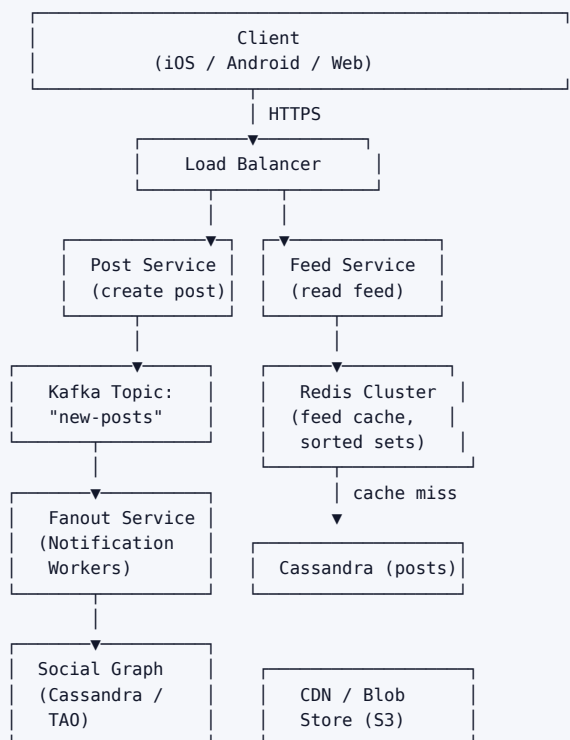
### Feed Cache (Redis Sorted Set per user):

```

Key:      "feed:{user_id}"
Members: post_id
Scores:   timestamp (epoch ms) — higher = newer
TTL:      48 hours (evict inactive users)
Max size: 800 posts per user (trim older entries)

```

## 3.5 5. High-Level Architecture



Post creation flow: Post Svc → Cassandra + Kafka  
 Fanout flow: Kafka → Fanout Workers → Redis (write to follower feeds)  
 Feed read flow: Feed Svc → Redis cache → (miss) Cassandra

## 3.6 6. Deep Dives

### 6.1 Fanout Strategies

| Strategy                      | How                                                                        | Pros                                     | Cons                                                                                   | Best for                               |
|-------------------------------|----------------------------------------------------------------------------|------------------------------------------|----------------------------------------------------------------------------------------|----------------------------------------|
| <b>Fanout-on-Write (Push)</b> | On post creation, write <code>post_id</code> to each follower's feed cache | Feed reads $O(1)$ — just read from cache | Write amplification: 1 post → $N$ writes; celebrity (10M followers) = 10M cache writes | Average users                          |
| <b>Fanout-on-Read (Pull)</b>  | On feed request, fetch posts from each followee, merge, sort               | No write amplification                   | Feed read $O(N \text{ followees})$ — slow for users following many people              | Celebrity accounts, inactive followers |



| Strategy                    | How                                                                                        | Pros                         | Cons               | Best for                              |
|-----------------------------|--------------------------------------------------------------------------------------------|------------------------------|--------------------|---------------------------------------|
| <b>Hybrid (recommended)</b> | Push to regular followers; pull for celebrities at read time; don't push to inactive users | Balances read and write load | More complex logic | <b>Large-scale production systems</b> |

### Key Decision: Hybrid fanout.

Implementation:

- › Define "celebrity" as users with > 1M followers.
- › On post write: fanout-on-write for non-celebrities; for celebrities, mark the post in a "celebrity posts" index only.
- › On feed read: read user's pre-built feed from Redis (push-mode posts) + fetch latest celebrity posts from celebrity index + merge and deduplicate.
- › Skip fanout for users who haven't been active in 7 days (check last\_active timestamp).

## 6.2 Celebrity Problem (Hotspot)

A celebrity with 10 million followers posts → need 10 million cache writes within seconds.

**Mitigation:**

1. **Async fanout workers:** Put the fanout job on Kafka with 10,000 partitions. Workers consume in parallel — 10M writes spread across workers.
2. **Pull for celebrity content:** Don't write celebrity posts to follower feeds. At read time, the top-K celebrities a user follows are fetched separately and injected into the feed.
3. **Pre-fetch:** For celebrities, replicate their latest posts to a separate, highly-cached "celebrity feed" shard. All followers read from the same shard (CDN-like effect).

## 6.3 Feed Ranking

Simple timeline: sort by score = timestamp.

ML-ranked feed (Facebook/Instagram model):

```
score = f(recency, engagement_velocity, relationship_strength, content_type)

engagement_velocity = likes_per_hour / avg_likes_per_hour_for_author
relationship_strength = interaction_history(viewer, author)
```

**Interview takeaway:** In the interview, acknowledge ranking but defer details. Say: "We'd train an ML model offline, serve predictions via a Feature Store + low-latency inference service. The feed service re-ranks the top 200 candidates before returning the top 20."

## 6.4 Pagination

**Cursor-based pagination (not offset):**

- › Offset pagination (LIMIT 20 OFFSET 100) breaks when new posts are inserted — items shift, causing duplicates/skips.
- › Cursor = encoded last-seen post timestamp + post\_id.
- › Redis: ZRANGEBYSCORE feed:{user\_id} -inf <cursor\_score> LIMIT 0 20.
- › Cursor makes the feed stable even as new posts arrive.

## 3.7 7. Bottlenecks & Scaling

**Feed cache size:**

- › 10 TB for active users — run 50+ Redis nodes (200 GB each) in a cluster.

- › Use consistent hashing for user → Redis shard mapping.
- › Evict feeds of users inactive for 7 days to save memory.

#### Cassandra post storage:

- › Partition by author\_id → hot partition if one author posts very frequently.
- › Mitigate: add bucket to partition key: (author\_id, bucket) where bucket = created\_at / 30 days.

#### Social graph at scale:

- › Facebook built TAO (The Associations and Objects) — a custom graph store on top of MySQL + Mem-cache.
- › At smaller scale: Cassandra follows\_by\_follower + follows\_by\_followee tables work well.

#### Read path latency breakdown (target < 200 ms):

```
Redis feed read:      5 ms
Celebrity post fetch: 20 ms (parallel)
User profile hydration: 10 ms (from user cache)
Ranking re-score:     15 ms
Network + serialization: 10 ms
Total:                ~60 ms (well within 200 ms budget)
```

## 3.8 8. Trade-offs Summary

| Decision           | Choice Made                        | Alternative             | Reason                                                 |
|--------------------|------------------------------------|-------------------------|--------------------------------------------------------|
| Fanout strategy    | <b>Hybrid (push+pull)</b>          | Pure push or pure pull  | Balances write amplification vs read latency           |
| Feed storage       | <b>Redis sorted sets</b>           | DB query at read time   | Sub-10ms reads; DB can't handle 87K reads/sec directly |
| Feed ordering      | <b>Reverse-chronological (MVP)</b> | ML-ranked feed          | Simpler; ranked feed is a layer on top                 |
| Pagination         | <b>Cursor-based</b>                | Offset-based            | Stable under inserts; no duplicate/skip on new posts   |
| Post DB            | <b>Cassandra</b>                   | PostgreSQL (sharded)    | Write throughput; natural time-series clustering       |
| Celebrity handling | <b>Pull at read time</b>           | Fanout to all followers | Avoids 10M+ write storms per celebrity post            |

## 4 4. Design a Chat System (WhatsApp / Messenger)

### 4.1 1. Requirements

#### Clarifying questions to ask the interviewer:

- › 1:1 chat only, or also group chat? What is the max group size?
- › Do we need real-time delivery (online) and push notifications (offline)?
- › Media support (images, video, voice)?
- › Do messages need end-to-end encryption?
- › Message history: how far back? Searchable?
- › Read receipts (sent / delivered / read)?
- › Online presence ("last seen")?

#### Functional Requirements:

1. 1:1 messaging with real-time delivery.

2. Group messaging (up to 500 members per group).
3. Online/offline status with "last seen" timestamp.
4. Read receipts: sent → delivered → read.
5. Push notifications for offline users.
6. Message history persisted; searchable.
7. Media: images, files (< 100 MB each).

#### Non-Functional Requirements:

- › 500 million DAU; 100 billion messages/day.
- › Message delivery latency < 500 ms (online users).
- › High availability: 99.99%.
- › Messages delivered exactly once and in order.
- › End-to-end encryption support (Signal Protocol) — note architecture impact.

## 4.2 2. Capacity Estimation

### Assumptions:

- › 500 million DAU.
- › Each user sends 40 messages/day → 20 billion messages/day (1:1 + group).
- › Average message size: 100 bytes (text).
- › Average group has 10 members → group message creates 9 additional deliveries.
- › Media: 5% of messages are media → 1 billion media messages/day.

#### Message write QPS:

20,000,000,000 / 86,400 ≈ 231,481 msg/sec ≈ ~230,000 messages/sec  
 Peak (3×): ≈ ~700,000 messages/sec

#### Storage:

Text messages: 20B/day × 100 B = 2 TB/day  
 Media: 1B/day × 1 MB avg = 1 PB/day → S3/GCS  
 Metadata: 20B/day × 50 B = 1 TB/day  
 Total text+meta ≈ 3 TB/day → 1.1 PB/year (text only)

#### Concurrent WebSocket connections:

500M DAU × 30% online simultaneously = 150M concurrent connections  
 Each connection: 65KB OS overhead  
 Total: 150M × 65KB ≈ 9.75 TB RAM just for connections  
 → Need ~100,000 chat servers @ 1,500 connections each  
 (In practice: use event-driven servers; Nginx/Go handles 100K connections per server)  
 → 1,500 servers (100K connections each)

## 4.3 3. API Design

```
# WebSocket connection (real-time channel)
ws://chat.example.com/ws?token=<auth_token>

# WebSocket message format (client → server)
{
  "type": "message",
  "msg_id": "client-generated-uuid", // for deduplication
  "to": "user:5678", // or "group:9999"
  "content": "Hey!",
  "media_url": null,
  "timestamp": 1718000000000 // client time ms
}
```

```

}

# WebSocket message format (server → client, push delivery)
{
  "type":      "message",
  "msg_id":    "server-assigned-uuid",
  "from":      "user:1234",
  "to":        "user:5678",
  "content":   "Hey!",
  "server_ts": 1718000000050,      // server timestamp (canonical ordering)
  "seq":       100034              // sequence number in conversation
}

# ACK (client → server, delivery acknowledgement)
{ "type": "ack", "msg_id": "...", "ack_type": "delivered" }
{ "type": "ack", "msg_id": "...", "ack_type": "read" }

# REST: Send message (fallback for HTTP clients)
POST /api/v1/messages
Request: { "to": "user:5678", "content": "Hey!" }
Response: { "msg_id": "...", "server_ts": 1718000000050 }

# REST: Fetch message history
GET /api/v1/conversations/{conversation_id}/messages?before=<msg_id&limit=50
Response: { "messages": [...], "has_more": true }

# REST: Upload media
POST /api/v1/media/upload
Response: { "media_url": "https://cdn.example.com/m/abc.jpg", "expires_in": 86400 }

# REST: Get online status
GET /api/v1/users/{user_id}/presence
Response: { "online": false, "last_seen": "2026-06-14T09:55:00Z" }

```

## 4.4 4. Data Model

### Storage choices:

- › **Messages:** HBase / Cassandra — write-heavy time-series; partition by conversation, cluster by time.
- › **Conversations/Groups:** Cassandra or DynamoDB.
- › **User data:** PostgreSQL (low write, complex queries).
- › **Presence (online status):** Redis (ephemeral, TTL-based).
- › **Message queue for offline delivery:** Kafka or internal queues.
- › **Media:** S3 / GCS (blob).

### Messages Table (HBase / Cassandra):

```

messages

conversation_id  TEXT      PARTITION KEY    -- "conv:{user1}_{user2}" or "group:{id}"
seq_num         BIGINT    CLUSTERING KEY DESC -- monotonically increasing per conversation
msg_id          UUID      -- globally unique
sender_id       UUID
content         TEXT
media_url       TEXT
msg_type        TEXT      -- "text", "image", "video", "system"
status          TEXT      -- "sent", "delivered", "read"
created_at      TIMESTAMP

```

### Conversations Table:

conversations

```

conversation_id  TEXT    PRIMARY KEY
participants     LIST<UUID>
group_name       TEXT    (null for 1:1)
last_msg_id      UUID
last_msg_at      TIMESTAMP

```

### User Conversations Index (reverse lookup):

user\_conversations -- "what conversations is user X in?"

```

user_id          UUID    PARTITION KEY
conversation_id  TEXT    CLUSTERING KEY
last_msg_at      TIMESTAMP (for sorting)
unread_count     INT

```

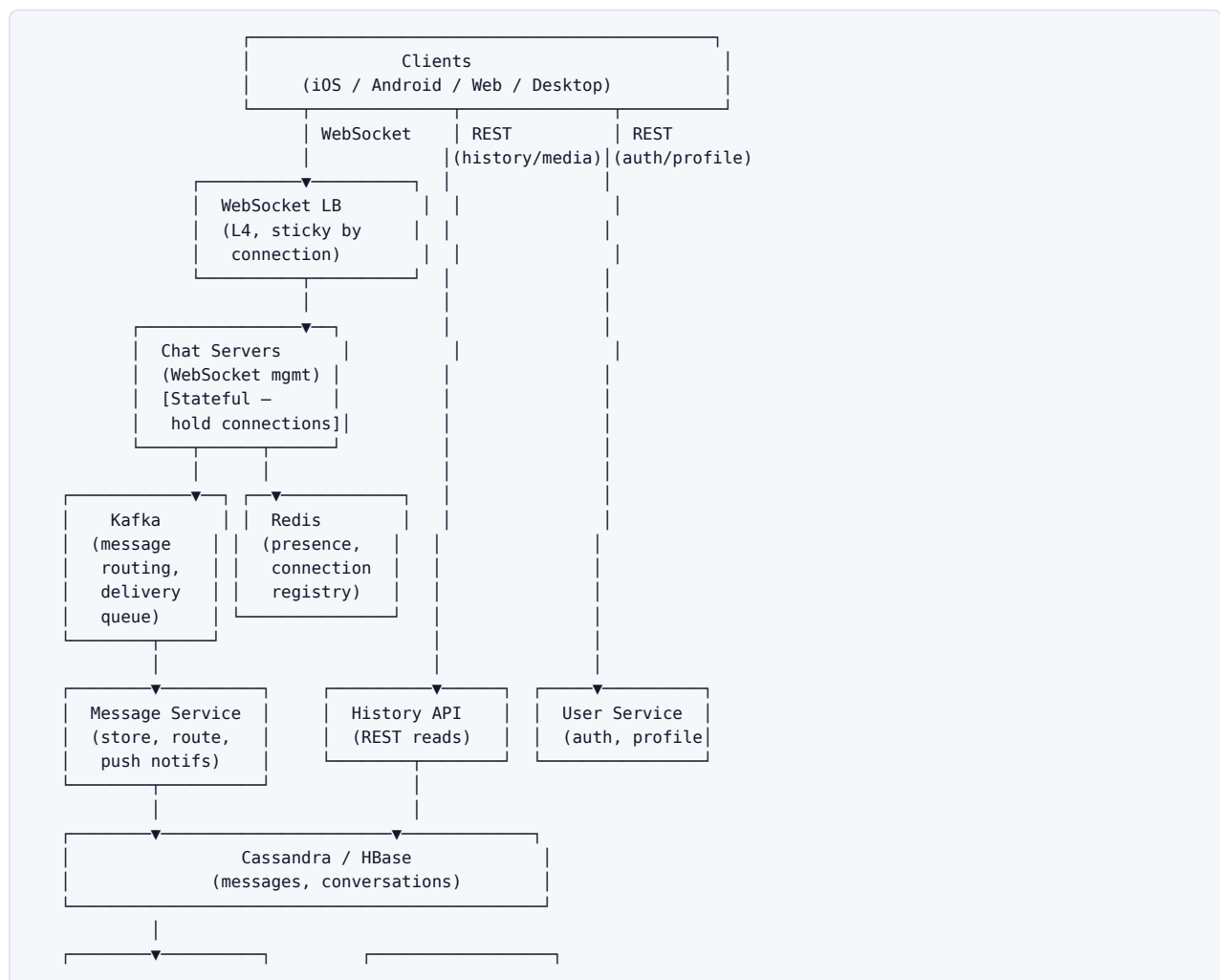
### Presence Table (Redis):

```

Key:  "presence:{user_id}"
Value: { "online": true, "last_heartbeat": 1718000000000 }
TTL:  30 seconds (refreshed by heartbeat every 10s)

```

## 4.5 5. High-Level Architecture



Push Notif Svc  
(APNs, FCM)

S3 / GCS  
(media blobs)

### Message delivery flow (1:1, both online):

1. Sender client sends message over WebSocket to **Chat Server A**.
2. Chat Server A writes message to **Kafka** (topic: messages).
3. **Message Service** consumes from Kafka, persists to Cassandra.
4. Message Service looks up which **Chat Server** the recipient is connected to (via Redis connection registry: "ws\_server:{user\_id}" → "chat-server-B").
5. Message Service pushes message to **Chat Server B** (via internal pub/sub).
6. Chat Server B delivers to recipient's WebSocket.
7. Recipient client sends ack: delivered → Chat Server B → Message Service → Cassandra (update status) → Kafka → Chat Server A → sender gets double-tick.

## 4.6 6. Deep Dives

### 6.1 WebSocket Connection Management

#### Why WebSocket instead of HTTP polling?

| Method                   | How                                           | Pros                         | Cons                                                 |
|--------------------------|-----------------------------------------------|------------------------------|------------------------------------------------------|
| Short polling            | Client polls every N seconds                  | Simple                       | Wasted requests; high latency (up to N sec)          |
| Long polling             | Server holds request until message or timeout | Fewer round trips            | Still HTTP overhead; server holds threads            |
| <b>WebSocket</b>         | Full-duplex TCP connection                    | True real-time; low overhead | Stateful — servers must track who is connected where |
| SSE (Server-Sent Events) | Server pushes, client uses XHR for sends      | Simpler than WS              | Unidirectional; need separate channel for sends      |

**Key Decision: WebSocket for all real-time delivery.**

**Connection registry in Redis:**

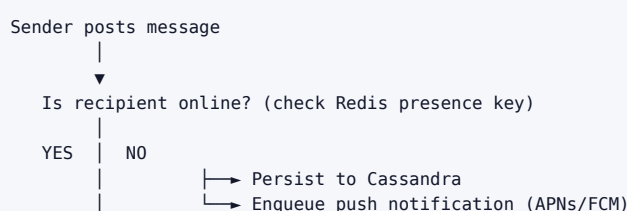
```
SET "conn:{user_id}" "chat-server-47" EX 300
```

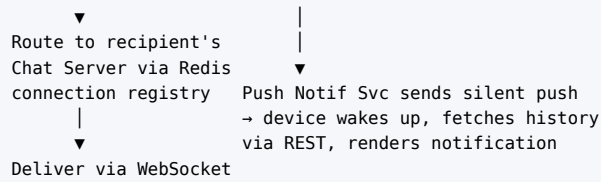
- › Refreshed every 60s by Chat Server via heartbeat.
- › If key expires (server crash), user treated as offline → push notification path.

#### Load balancing WebSocket connections:

- › L4 load balancer (not L7) — connections are long-lived TCP; no HTTP termination per request.
- › Use consistent hashing on user\_id for connection routing (optional but reduces cross-server lookups).

### 6.2 Message Delivery: Online vs Offline



**Message deduplication:**

- › Client generates a UUID (`client_msg_id`) before sending.
- › Server stores `client_msg_id` → `server_msg_id` mapping for 24 hours.
- › If retry received with same `client_msg_id`, return existing `server_msg_id` (idempotent).

**6.3 Group Chat Delivery****Small groups ( $\leq 500$  members):**

- › Fanout on message write: write one message record, but create one delivery record per member.
- › Alternative: write one message, each member's client polls for group updates using a `last_read_seq` cursor.

**Recommended for  $\leq 500$  members:**

```

Message write → Kafka
Kafka consumer → write 1 message to messages table
                  → write N "unread" records to user_conversation table (one per member)
Online members → WebSocket push
Offline members → Push notification
  
```

**Very large groups ( $> 500$ , e.g., broadcast channels):**

- › Switch to fanout-on-read: message is in one place, members' clients pull new messages using `seq_num` cursor.
- › Similar to news feed pull model.

**6.4 Message Ordering and Sequence Numbers**

**Problem:** Two users send messages simultaneously. Who goes first?

**Solution:** Sequence number per conversation (not global).

- › Each conversation has a monotonic `seq_num` counter stored in Redis or a coordination service (like Zookeeper).
- › When Message Service processes a message, it atomically increments `seq_num` for that `conversation_id` and assigns it to the message.
- › Messages displayed in `seq_num` order on all clients.

**Clock skew:** Client timestamps cannot be trusted. Always use server-assigned `seq_num` for ordering. Client timestamp only shown as display time.

```

INCR "seq:{conversation_id}" → returns 100034 (atomic in Redis)
Store message with seq=100034
  
```

**6.5 Online Presence****Heartbeat protocol:**

- › Client sends WebSocket heartbeat every 10 seconds.
- › Chat Server updates Redis: `SET "presence:{user_id}" 1 EX 30`.
- › Key expires in 30s if no heartbeat → user considered offline.

**Displaying presence:**

- › When user A opens a chat with user B: fetch GET "presence:{user\_id\_B}".
- › "Last seen": stored as last\_seen:{user\_id} on heartbeat stop (WebSocket disconnect or timeout).

#### Scale challenge:

- › 500M DAU × heartbeat every 10s = 50M presence writes/sec to Redis.
- › **Optimization:** Only send heartbeat if the user is actively viewing a conversation. Idle app = slower heartbeat (30s). Closed app = no heartbeat.
- › Use presence fanout sparingly: only push presence updates to contacts who have a conversation open with that user.

## 6.6 Read Receipts

**States:** sent → delivered → read.

```
sent:      Message reached the chat server (ack from server to sender).
delivered: Message delivered to recipient's device (ack from recipient device).
read:      Recipient opened the conversation (client sends "read" ack).
```

#### Implementation:

- › Each message has a status field in Cassandra.
- › Recipient sends {"type":"ack","msg\_id":"...","ack\_type":"read"} over WebSocket.
- › Chat Server → Message Service → update Cassandra status → forward read receipt to sender's Chat Server → sender gets green double-tick.

#### Group read receipts:

- › Store read\_by: Map<user\_id, timestamp> per message in Cassandra.
- › "Delivered to all" = all members' devices have acknowledged delivered.
- › Show "Read by X, Y, Z" as clients send individual read acks.

## 4.7 7. Bottlenecks & Scaling

### 150 million concurrent WebSocket connections:

- › Each Chat Server handles ~50K–100K connections (Go or Node.js event loop).
- › Need ~1,500–3,000 Chat Servers.
- › Chat Servers are stateful but share no business logic — scale horizontally, no coupling.

#### Kafka throughput (230K messages/sec):

- › Partition by conversation\_id (ensures ordering per conversation).
- › 230K msg/sec × 1 KB = 230 MB/s — Kafka handles this easily with dozens of brokers.
- › Message Service consumer group: scale consumers to match Kafka partition count.

#### Cassandra message storage (3 TB/day):

- › Partition by conversation\_id ensures messages in same conversation are co-located.
- › Hot conversation: one partition gets hammered → add bucket (time shard) to partition key.
- › 3 TB/day × 365 = ~1.1 PB/year. Cassandra compresses ~3:1 → ~370 TB/year of disk.
- › TTL old messages: archive messages > 1 year to cold storage (S3 Glacier), keep last 1 year hot.

#### Media delivery:

- › Don't send media over WebSocket — too large.
- › Client uploads directly to S3 (pre-signed URL from Media Service).
- › Sends media URL in the chat message. Recipients download from CDN-fronted S3.

#### Push notification scale:

- › 500M users. At any time, 70% offline.



- › Surge: celebrity group sends a message → 1M push notifications simultaneously.
- › Use async workers + APNs/FCM batch APIs. Rate limit per APNs connection (use multiple connections).

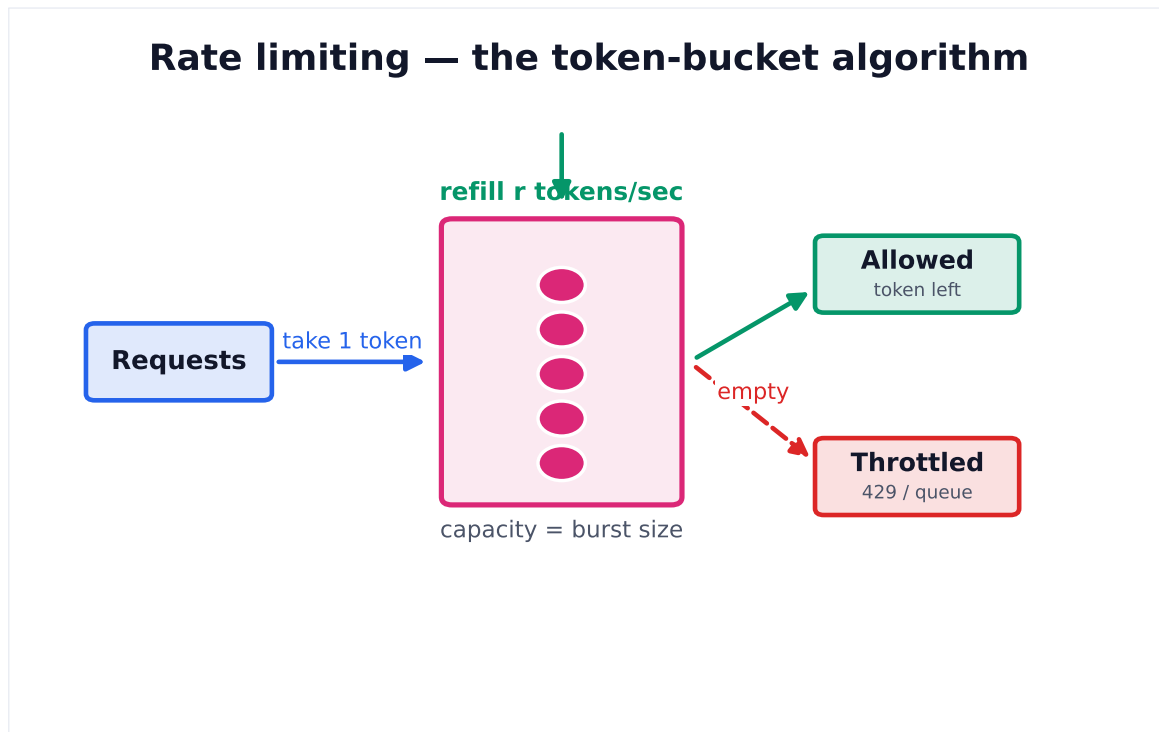
## 4.8 8. Trade-offs Summary

| Decision            | Choice Made                                           | Alternative                   | Reason                                                        |
|---------------------|-------------------------------------------------------|-------------------------------|---------------------------------------------------------------|
| Real-time transport | <b>WebSocket</b>                                      | Long polling / SSE            | Full-duplex; lowest latency; widely supported                 |
| Message storage     | <b>Cassandra (conv-partitioned)</b>                   | MySQL (sharded)               | Write throughput; time-series access pattern; no joins needed |
| Connection registry | <b>Redis (TTL-based)</b>                              | Consistent hash routing       | Fast lookup; auto-evicts stale connections on TTL expiry      |
| Message ordering    | <b>Server-assigned seq_num (Redis INCR)</b>           | Vector clocks                 | Simple; sufficient for single-conversation ordering           |
| Group delivery      | <b>Fanout to members (&lt; 500) / pull (&gt; 500)</b> | Always pull                   | Balances latency vs write amplification by group size         |
| Presence heartbeat  | <b>10s client heartbeat, 30s TTL</b>                  | Server-push disconnect events | Handles silent disconnects (network failure, app killed)      |
| Read receipts       | <b>Per-message status in Cassandra</b>                | Separate receipt table        | Avoids join; co-located with message data                     |
| Media               | <b>S3 + CDN (URL in message)</b>                      | In-message blob               | Media over WebSocket is impractical at scale                  |

*End of Part 1 — Foundational System Design Problems.*

## 5 Visual Reference

A set of figures distilling the key quantitative intuitions of this topic.



**Figure 1.** A bucket refills at a steady rate up to a capacity. Each request spends a token; an empty bucket throttles. This smooths traffic while allowing short bursts.